1 - Exam facts, and how to attack a question

Read this page first on exam day. It is worth more than any single fact.

Questions	Time	Pass mark	Scenarios
60	120 min (2 min each)	720 / 1000	4, drawn from 6
D1 Agentic	**D2 Tools + MCP**	**D3 Claude Code**	**D4 Prompts / D5 Context**
27% (~16 q)	18% (~11 q)	20% (~12 q)	20% (~12 q) / 15% (~9 q)

Each question states **how many answers to select** - read that line. No penalty for guessing, so never leave one blank. If a question takes more than 3 minutes, mark it and come back.

The 30-second method

1	Read the **last sentence** of the question first. That is what is actually asked.
2	Note the signal words (page 2). They usually name the answer type.
3	Remove any answer describing a feature that **does not exist**.
4	Remove any answer blaming a part the question says is **working correctly**.
5	From what is left, choose the **weakest fix that still meets the requirement**.

The strength ladder - choose the lowest level that meets the requirement

Level	Fix	Use when
1	Better **tool descriptions**	Model picks the wrong tool; descriptions are short or similar
2	**Few-shot examples** (2-4)	Behaviour is inconsistent in unclear cases
3	**Explicit criteria** in the prompt	Too many false alarms; current instruction is vague
4	**Programmatic enforcement** hooks, prerequisite gates, forced tool_choice	The rule must ALWAYS work
5	Separate classifier / ML system	Almost never the answer on this exam

Level 4 wins whenever the question mentions any of these

Money (refunds, payments, billing) - identity checks - policy limits or thresholds

The words **deterministic**, **guaranteed**, **must**, or **never**

Reason from the guide: prompt instructions have a **non-zero failure rate**. Sometimes is not acceptable when money is involved.

The five kinds of wrong answer

Too big a solution	Builds new systems before trying simple fixes. Signals: train a classifier, deploy a model, add a routing layer, build a service
Blames a working part	The question names the broken part. Any answer pointing elsewhere is wrong
Feature does not exist	Sounds confident, describes a flag or file that is not real (see page 2)
Solves another problem	Reasonable, but fixes something the question did not ask about
Moves work to people	Asks developers or customers to do more, or hides real problems

Never the right answer

Self-reported confidence used to decide when to **escalate**
Sentiment analysis used to judge how difficult a case is
A **bigger context window** used to fix inconsistent quality or attention dilution
Voting across several runs to filter findings - this hides real bugs
Reading the model's **text** to decide when the loop stops - use stop_reason
A fixed **iteration cap** as the main stopping method (fine as a safety limit)
Hiding errors by returning empty results as success
Stopping the whole workflow because one subagent failed
Guessing when a tool returns several matches - ask for another identifier

2 - Signal words, paths, and flags

Domain 3 is 20% of the exam and almost entirely memorisation. These are the cheapest points.

Signal words in the question

Words in the question	What to choose
most effective **first step**	The cheap simple fix (level 1-3), not the big one
deterministic, guaranteed, must never	Programmatic enforcement (level 4)
the logs show the coordinator...	The cause is named. Fix that part
minimal descriptions	Improve the tool descriptions
blocking, developers wait	Real-time API, not batch
overnight, next morning, weekly	Batch API is fine
inconsistent results, contradictory feedback	Split into smaller focused passes
false positives, developer trust	Write explicit criteria
hangs, waiting for input	-p / --print
every developer when they clone the repo	Project scope: .claude/... in the repository
spread throughout the codebase	.claude/rules/ with glob patterns
dozens of files, architectural decisions	Plan mode
single-file fix with a clear stack trace	Direct execution

File paths

Personal instructions - **NOT shared**	~/.claude/CLAUDE.md
Project instructions - shared	.claude/CLAUDE.md or CLAUDE.md in the root
Folder instructions	CLAUDE.md inside that folder
Rule files by topic	.claude/rules/
Shared slash commands	.claude/commands/
Personal slash commands	~/.claude/commands/
Shared skills	.claude/skills/
Personal skills	~/.claude/skills/
Shared MCP servers	.mcp.json
Personal MCP servers	~/.claude.json

Rule: anything starting with **~/** is personal and is NOT shared through version control.

Frontmatter keys - do not mix these two

SKILL.md (in .claude/skills/)	.claude/rules/ files
context: fork run in a separate context, keeping long output out of the main conversation **allowed-tools** limit which tools the skill may use **argument-hint** ask the developer for a missing parameter	**paths:** glob patterns. The rule loads only when you edit a matching file Example: paths: ["terraform/**/*"] paths: ["**/*.test.tsx"]

paths: is NOT a SKILL.md key. Mixing these two sets is the most common mistake in Domain 3.

Commands and flags

See which memory files are loaded	/memory
Reduce context in a long session	/compact
Continue a named session	--resume <name>
Branch from one shared analysis	fork_session
Non-interactive mode in CI - fixes hanging	-p or --print
Output a script can read	--output-format json
Force output to follow a schema	--json-schema
Noisy discovery without filling the context	Explore subagent

These DO NOT EXIST - they appear as wrong answers
--batch CLAUDE_HEADLESS=true --no-interactive
.claude/config.json with a commands list
Redirecting stdin from /dev/null as the "correct approach"

3 - Domain 1: Agentic Architecture (27%)

The biggest domain. About 16 of the 60 questions.

The agentic loop

The loop is controlled by **stop_reason**. Nothing else.
"tool_use" -> run the tools, add results to history, repeat
"end_turn" -> stop
All six values: end_turn, max_tokens, stop_sequence, tool_use, pause_turn, refusal
Every tool_use needs one tool_result with the same **tool_use_id**
Send all results from one Claude message back in **ONE** user message

Three wrong ways to stop the loop
1. Reading Claude's text for a phrase like "I have completed"
2. A loop counter as the MAIN stopping method (as a safety limit it is fine)
3. Checking whether the response contains text - text and tool calls appear together

Subagents

Started with the **Task tool**
The coordinator's **allowedTools must include "Task"** - without it, no delegation is possible
Subagents receive **NOTHING** automatically. No history, no memory. Put it all in the prompt
Parallel = several Task calls in **ONE response**
fork_session = branches from ONE shared analysis, to compare approaches
All messages go **through the coordinator**. Subagents never talk to each other
AgentDefinition = each subagent's description, system prompt, and tool restrictions

The failure to remember
Coverage is incomplete BUT every subagent worked correctly
= the coordinator's **task decomposition was too narrow**
Any answer blaming a downstream agent is wrong.

Making a rule that always works

Programmatic enforcement	Prompt-based guidance
Hooks, prerequisite gates	System prompt, few-shot examples
Always works	Fails sometimes
Money, identity checks, policy limits, compliance	Style, preferences, judgement

The two hook types

PostToolUse	Changes tool **RESULTS** before the model reads them. Use for normalising different data formats (Unix time / ISO 8601 / numeric codes)
Interception hook	Blocks an outgoing tool **CALL**. Use for blocking refunds over $500 and redirecting to escalation

Splitting up work

Prompt chaining fixed steps	The work is predictable. Large code review = **one pass per file + one pass across files**
Dynamic decomposition steps depend on findings	The work is open-ended. Example: add tests to a legacy codebase

A bigger context window does **NOT** fix attention dilution.

Sessions

Earlier context is mostly still correct	--resume, and tell it which files changed
Earlier tool results are **stale**	Start a NEW session with a written summary

4 - Domain 2: Tool Design and MCP (18%)

About 11 of the 60 questions.

Tool descriptions - the core fact

Tool **descriptions** are how the model chooses a tool. Improve them **first**.
A description must contain: **inputs**, **example queries**, **edge cases**, and **when to use it instead of a similar tool**.
Overlapping tools -> **rename** and rewrite (analyze_content -> extract_web_results)
One tool doing three jobs -> **split** it, with defined inputs and outputs
Check the **system prompt** for words that create unwanted tool links
Improve MCP descriptions, or the agent prefers built-in Grep over a better MCP tool

Error responses

Category	Example	Agent should
transient	Timeout, service down	Retry
validation	Bad input	Fix the input, retry
business	Breaks a policy	Explain to the user. Do NOT retry
permission	Not allowed	Escalate or explain

Every error returns: **errorCategory** + **isRetryable** + a readable message
Business rules also need **retriable: false** and a customer-friendly explanation
An **access failure is NOT an empty result**. A timeout may need a retry; zero matches is a success
Subagents fix transient failures **themselves**. They only report what they cannot fix
When reporting: failure type + what was tried + partial results + alternatives

tool_choice

auto	May reply with **TEXT** instead of calling a tool. Never the fix when it returns text
any	Must call a tool, chooses which. Use when the document type is unknown
{"type":"tool","name":...}	Must call THAT tool. Use to force extract_metadata to run first
none	Cannot use tools

auto may talk - any must act - forced picks the actor.

Tool distribution

18 tools is bad. 4-5 is good. More tools = worse selection
An agent with tools outside its job **will misuse them**
Give a small cross-role tool for a common need (verify_fact for the synthesis agent)
Replace general tools with limited ones (fetch_url -> load_document)

MCP configuration

Shared team servers	.mcp.json (project, version-controlled)
Personal / experimental	~/.claude.json (user)
Keep secrets out of the repo	${GITHUB_TOKEN} environment variable expansion
MCP resources	Content catalogues: issue lists, doc structures, DB schemas. They stop the agent making exploratory calls
Standard integration (Jira)	Use a **community** server. Build custom only for your own workflows

Built-in tools

Grep	Searches INSIDE files - function callers, error messages, imports
Glob	Matches FILE PATHS - **/*.test.tsx
Edit fails on unclear text	Use **Read + Write** instead
Exploring a codebase	Grep to find entry points, then Read to follow imports. Do not read everything

5 - Domain 4: Prompts and Structured Output (20%)

Your weakest domain on the day-1 test. Read this page every day.

Schema design

A **required** field makes the model **INVENT** a value. Make it **optional and nullable**
Enum + **"other"** + a detail string, for categories that may grow
Enum value **"unclear"** for ambiguous cases
Tool use + schema removes **SYNTAX** errors. It does NOT remove **SEMANTIC** errors
calculated_total next to **stated_total** - catches totals that do not add up
conflict_detected - boolean for contradictory source data
detected_pattern - to analyse which findings developers dismiss
Put format normalisation rules in the **prompt**, next to the strict schema

Retries

Retry WORKS	Retry DOES NOT WORK
Wrong format (dates, currency)	The information is **not in the document**
Wrong output structure	It is only in a document you did not supply

A retry request contains: the **document** + the **failed extraction** + the **specific validation errors**.

Message Batches API

Cost	**50% cheaper**
Time	**Up to 24 hours**
Speed guarantee	**NONE**
Matching results	**custom_id** (never by position)
Multi-turn tool calling	**Not supported**
Good for	Overnight reports, weekly audits, nightly test generation
Bad for	Anything where a person is waiting (pre-merge, pre-commit)
Failures	Resubmit **only** the failures, found by custom_id. Split long documents
SLA maths	24-hour window + 30-hour SLA -> submit every **4 hours**

Prompts and criteria

Specific categories work. **"Be conservative"** does not
Confidence-based filtering does **not** improve precision
A category with many false alarms damages trust in the good ones -> **turn it off** while you fix it
Severity levels need **concrete code examples** for each level
Few-shot: 2 to 4 examples, aimed at the **unclear** cases, not the obvious ones
Few-shot lets the model **generalise** to new situations, not just copy

Review architecture

Self-review is weak: the model remembers its own reasoning -> use an **independent instance**
Large review = **one pass per file + one separate pass across files**
A bigger context window does **NOT** fix attention dilution
Voting across runs **hides real bugs**
Confidence IS allowed next to each finding, to route reviewer attention

6 - Domain 5: Context and Reliability (15%)

Only 15%, but it appears in four of the six exam scenarios.

Three context problems and their fixes

Problem	Fix
Progressive summarization loses numbers, dates, and what the customer expected	A **"case facts" block** in every prompt, OUTSIDE the summarised history
Lost in the middle - the model handles the start and end well, misses the middle	Key findings at the **BEGINNING**. Clear section headings for the details
Tool results fill the context - 40+ fields when only 5 matter	**Trim to the useful fields BEFORE** they enter the context

Escalation

Escalate when	NEVER escalate based on
The customer **asks** for a human	**Sentiment** - it does not predict difficulty
There is a **policy gap** (not just a hard case)	The model's own **confidence score**
No progress is possible	

The trap - these two look the same
Customer **explicitly asks** for a human -> escalate **IMMEDIATELY**, do not investigate first
Customer is **frustrated** but you can help -> offer help, escalate only if they ask again
Several matching customers -> **ask for another identifier**. Never guess.

Errors between agents

Report four things: **failure type + what was tried + partial results + alternatives**
Never: a generic message like "search unavailable"
Never: hiding the error by returning empty results as success
Never: stopping the whole workflow because one agent failed
Never: treating an access failure and a valid empty result as the same thing
Synthesis output should mark which topics have **coverage gaps**

Long codebase exploration

Symptom of **context degradation**	The model talks about "**typical patterns**" instead of the specific classes it found
Fix	**Scratchpad files** - save key findings, read them later
Also	Subagent delegation, phase summaries, /compact
Crash recovery	**Manifests** - each agent exports state; the coordinator loads it on resume

Human review and confidence

97% average accuracy can hide one broken document type
Analyse accuracy **by document type and by field** before reducing human review
Stratified random sampling of HIGH-confidence extractions - that is the group you are about to stop checking
Confidence is usable **only when calibrated with labelled validation sets**, and only for routing review
Route to a human: low confidence, or the source is ambiguous or contradicts itself

Sources and provenance

Attribution is lost during **summarisation**, when claim-source links are not preserved
Subagents output **claim-source mappings**: source URL, document name, relevant excerpt
Subagents must include **publication dates**, or old and new numbers look like a contradiction
Two credible sources disagree -> **record BOTH with their sources**. Never pick one
Separate **well-established** findings from **contested** ones in the report
Format by content type: financial data = **tables**, news = **prose**, technical = **lists**